Git Version Control System Locally

Table of Contents

Part 01 – Installation and Configuration 3

Part 02 – Building and Tracking Apps with Git 6

Part 03 - Branching 9

Part 04 – Installing Nodemon 11

Part 05 – Merging the branches 12

Part 06 – Typical Git commands 13

Part 07 – Advanced Git Commands 14

Part 08 – Git Tagging 19

Part 09 – VS Code Source Control 21

Part 10 – Remote Repository 25

Appendix A – PAT Error 31

Appendix B – Showing the .git Folder in VS Code 32

Appendix C – Fixing Source Control Icon Not Refreshing 33

Appendix D – Fixing Source Control Icon Not Refreshing 34

Appendix E – Shorten the Linux Prompt 34

Appendix F – Common Errors 35

Appendix G – Postman Setup 35

Appendix H – git log VS git reflog 38

Beginner Tutorial for Git (in Ubuntu), VS Code and NodeJS

Git is a version control system that makes it easy to just "undo" a change to your project. It is like a time machine for code. Instead of saving multiple files like file01.js or file02.js, Git tracks every small change made to a single project folder (database). This allows developers to experiment with new features safely.

For a budding developer, it means that you can take risks, have a clear history of your learning progress, and collaborate with other engineers on shared codebases.

In this first day (of three) we will make sure that Git is installed, add our individual identity characteristics, setup a text editor and understand the basics of how Git works.

We will practice using Git features while building a small NodeJS application. If you are not familiar with Node then you will learn Node as a bonus. Node is JS based, so not much to learn if you have some JS knowledge, just configuration of the app itself.

This bootcamp does require a small bit of configuration in the beginning.

Part 01 – Installation and Configuration

This Part covers basic installation and configuration.

1. Install Git using `apt` or some other method. Once you have the software installed check the version. If no errors and a number like **2.54.0** comes back, then Git is installed and ready for the next step:

```
git --version
```

If you are on a Linux based system, you probably already have Git installed. You may just need to update to the latest version. The version 2.52.0 is the latest stable version as of this document's date.

2. Lets configure our credentials that Git will use. If you already have a GitHub.com account, just use that one:

```
axle@pc0480:~$ git config --global user.name "AxleB"
axle@pc0480:~$ git config --global user.email "axle.barr@skillsoft.com"
```

If you are only working on the local machine you could use --local instead of --global.
(use `git config --list` if you think that your credentials are already setup)

3. If you are comfortable with OS based text editors like Vim then you can skip this step. We will configure Git to call VS Code by default instead of the hard to use OS based one:

```
git config --global core.editor "code --wait"
```

The –-wait flag will keep the focus on the file you are working on.
So now all editing will be done in VS Code. If you prefer Notepad++ instead, you can use something like this:

```
git config --global core.editor "'C:/Program
Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -
noPlugin"
```

4. Create a working directory in Documents, and prepare it for Git, run these commands in order:

```
mkdir git-node
cd git-node
git init
```

```
axle@pc0482:~$ cd Documents
axle@pc0482:~/Documents$ mkdir git-setup
axle@pc0482:~/Documents$ cd git-setup
axle@pc0482:~/Documents/git-setup$ git init
hint: Using 'master' as the name for the initial branch. This default branch nam
e
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/axle/Documents/git-setup/.git/
axle@pc0482:~/Documents/git-setup$
```

5. To master Git, you need to understand the **Three Trees**: *the Working Directory,* the *Staging Area,* and the *History*. The current directory is now setup to work with git so just add a new file to it:

   - echo "Learning Git" > README.md

   After the above, type the `ls` command and you should see your readme file. Note you can also use the `touch` command for creating files.

6. Check the status of that file from #5. Remember this is the status according to git:

   - git status

```
axle@pc0482:~/Documents/git-setup$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

It's basically stating which branch you are on, so `master` in this case, there are no commits as yet but there is a file there that is not being tracked by git. They use the color red for this situation.

Note: although we are on the *Master* branch now, we will eventually change it to *Main* as that seems to be the way Git is moving forward.

7. Start tracking the file in red (add to the staging area):

```
git add README.md
```

Remember if you are on Linux or Mac, you must use proper casing. Using `git add` is the **staging** part of the three trees. If you have added files to the staging area but then made changes after staging, you **must** run `git add` again. The next step will be saving this file (commit).

8. Now we can commit to this tracking:

```
git commit -m "Added the Readme.md file"
```

You should see a message about one file being changed and one insertion

Note, you can run a `git status` at any time. If you did, the README file will not be red anymore at this point, but a commit still needs to be done.

The basic workflow is that you create a directory, immediately run `git init` against that directory and then continue to add and change files as you work. At some point do a `git add` Yes, there are flags to add **all** files, folders and changes at once: `git add --all`

Note, you can replace the `--all` with a *period* character.

9. Once you have added and then committed the README.md file, you have created your first *snapshot* of the project. We will continue working on the project but if at any time we wanted to start over the project, for any reason, we have points (snapshots) we can go back to and start over.

At this point you should have a folder called git-node with the README.md file in it, nothing else. Actually, there is a .git folder as well but it is hidden.

There are no zipped files for this part.

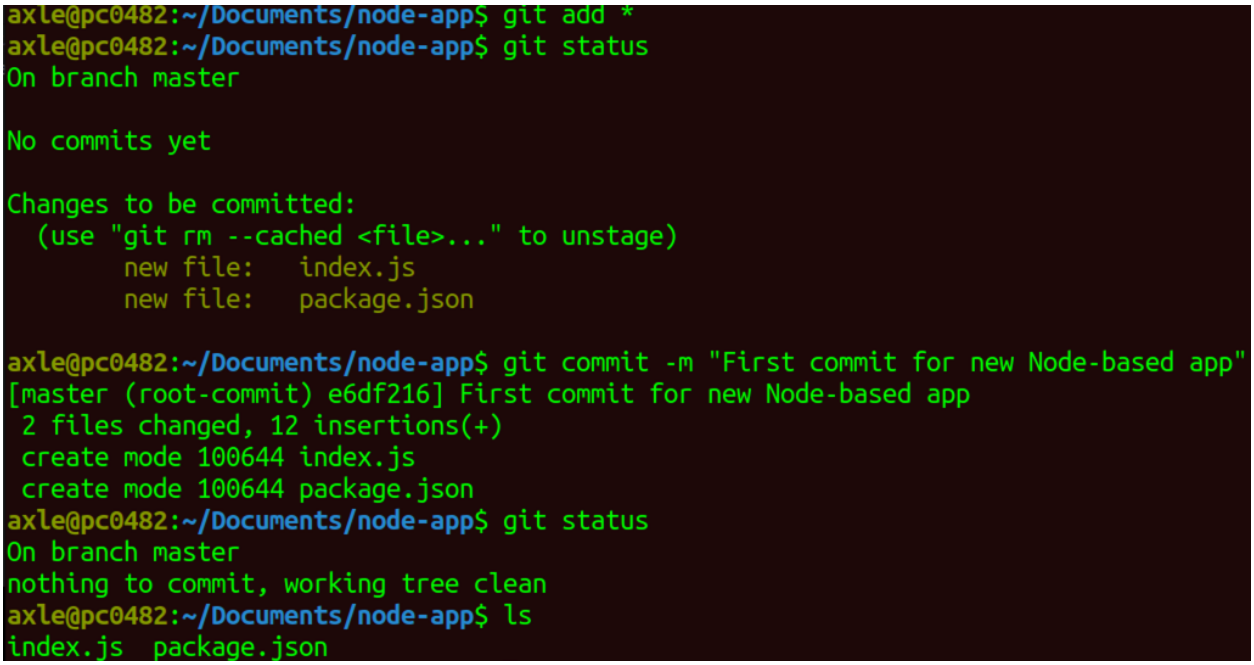
Part 02 – Building and Tracking Apps with Git

1. Continuing from Part01, you should be in a terminal widow pointing to the git-node directory. Let's turn to **NodeJS** for a minute and execute this command to initialize a Node application in the current directory:
   `npm init -y`

2. This command will give you a package.json file. This is a manifest of future packages for your Node app. Packages are just dependencies for your Node application, they power your app.

3. We will now add a file in the git-node directory. On Linux its `touch index.js`. This index file will act just like the `main()` method of Java, C++ and C#.

4. Now run the following git commands:
   ```
   git add *
   git status
   git commit -m "First commit for new Node-based app"
   git status
   ```



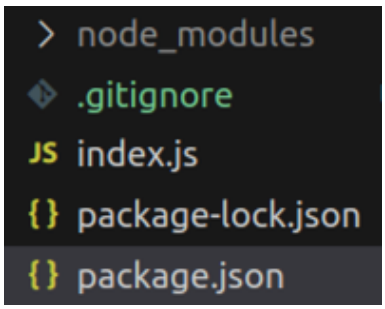
5. If you run the command `code .` it should open VS Code. Add the following JS code to your index file:

```
const express = require('express');
const port = 8000;
const app = express();
app.get('/', (request, response)=> response.send('hello from skillsoft'));
app.listen(port, function(){
      console.log("Listening " + port);
});
```

Note the Linux command to open VS Code is the word `code` followed by a *period* character. If this is the first time opening VS Code you may have to trust the git-node folder.

6. From VS Code, open a terminal window and type the command:
   `npm install express`

Your package.json file should have **express** in the *dependencies* area. Your folder structure should look like this:



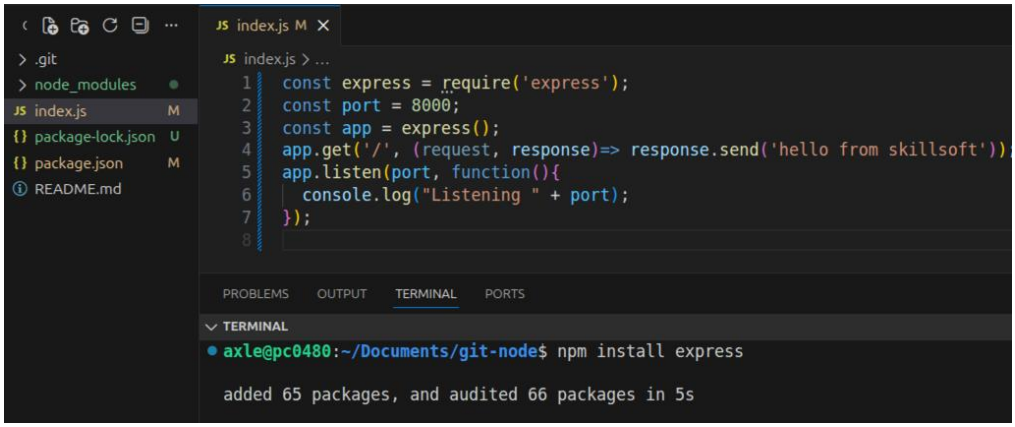
Note, to show hidden files, usually **Ctrl + H** will work on Ubuntu > v.20

To show hidden files in VS Code, refer to the Appendix B at the end of this document. In Linux any file/folder with a period in front of it makes that file or folder a *hidden* item.

You can now start using the embedded terminal that ships with VS Code.

7. So far we have used the "Big Three" (**add**, **status**, **commit**), which represents the basics of a local workflow. To move from "saving files" to truly "managing a project," the next steps should focus on **safety**, **collaboration**, and **history**. Mostly covered in a future bootcamp.

The image below gives you a good idea of what your VS Code window should look like. Yours may be slightly different but these are the three main windows for now. Later we will open GitHub Copilot Chat window bringing all windows in VS Code to four in total.



So, *file explore* to the left, *code editor* middle and *terminal* bottom.

8. Just one more configuration item and it is extremely important. You do not ever want Git to track the node_modules folder, NEVER! So lets remove this from Git tracking using the simple .gitignore file that is part of Git. If you do not have a .gitignore file, create it in the git-node directory using touch: `touch .gitignore`
   Do not forget the period in front of `gitignore`
   Note: .git is a folder .gitignore is a file, both are hidden. The .gitignore file must be in the root folder of your app.

9. Open .gitignore in VS Code and add the following lines:
   `node_modules`
   Save the changes in VS Code and now move to the terminal window where we will register these changes with Git

10. Run the command `git add --all`, then `git status` to make sure .gitignore has been added to the set of files that git checks. Finally run the command:
    `git commit -m "Added .gitignore file"`
    You can do a quick check using: `git check-ignore -v node_modules`

Part 03 - Branching

At this point we have a folder containing the beginnings of a Node application and that folder is being tracked by Git. That means that if we add a new file or change the code in an existing file, Git will know about these changes and a *status* will reveal them.

We will now create a Git branch add Router and routing to the app then we will merge the two branches if the routing part (feature) is successful. In app development terms, we are adding a new feature to the existing app. This can be done if checking out a project and are part of a team.

If you want to see the Node application in progress, just run the command `node index` in the terminal window. It will start a process. Then just go to *localhost:8000* and you will see the single line "hello from Skillsoft". Press **Ctrl + C** to end the process and return the terminal to normal.

1. Create a new branch with the following command:

```
git checkout -b feature-router
```

This command creates the branch and moves you onto that branch. You could also use: `git switch -c feature-router`. Return to previous branch: `git switch -`



Also, if you use a `git status` now, it should tell you that you are on the feature-router branch

2. Lets continue with Node, so add the Router package and instead of using `app.get()` for line 4, use `router.get()`:

```
const app = express();
const router = express.Router();
router.get('/', (request, response)=> response.send('hello from
skillsoft'));
app.listen(port, function(){
```

Note, Router is not an install, just a line of code.

3. Now just register router with the original app we created:

```
router.get('/', (request, response)=> response.send('hello from skillsoft'));
app.use('/', router);
app.listen(port, function(){
```

Test the app by restarting and checking the browser. You must restart for the browser to be aware of the changes. Later we will add a package that does this automatically.

4. Still in the index file, add a new route but this time we will POST something:

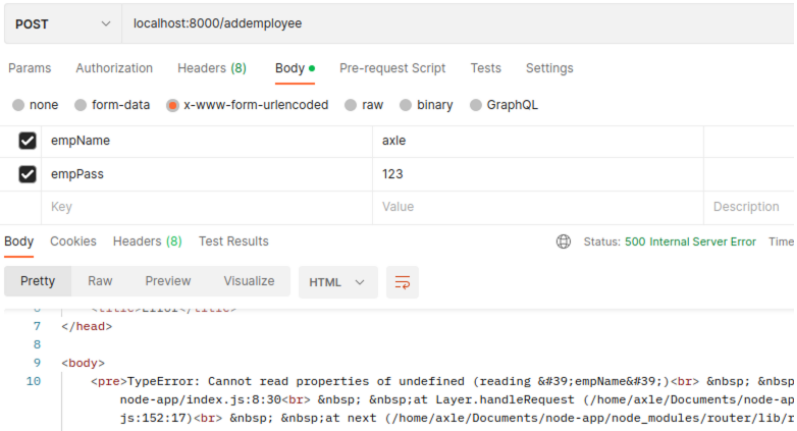
```
router.post('/addemployee', (request, response) => {
  let empName = request.body.empName;
  let empPass = request.body.empPass;
  response.end(`POST success, you sent ${empName} and ${empPass},
thanks!`);
});
```

```

```

This block can go just below the `router.get()` line of code but above the `app.use()` function. Remember to stop and then start the app again, in order to see the changes.

5. In order to test the POST route, you need special software like Postman. You could also add a browser plugin, but Postman is the industry standard:

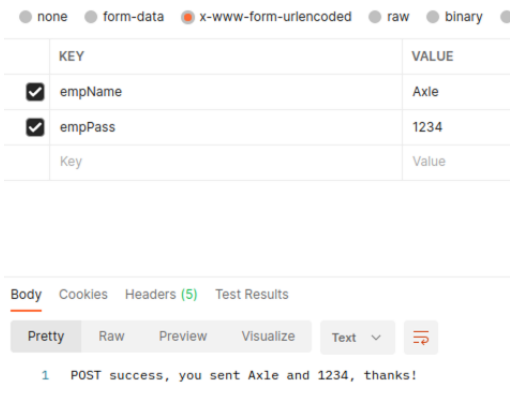


Notice that there is an error, we will fix this in #6 below

6. Express Router needs additional configuration in order for it to function, so add these two lines just before the first `get()` request:

```
const router = express.Router();
app.use(express.json());
app.use(express.urlencoded({extended:false}));
router.get('/', (request, response)=> response.send('hello
from skillsoft'));
router.post('/addemployee', (request, response)=>{
```

Test again once these lines are added. Both of these lines must go after app is created.



Part 04 – Installing Nodemon

Before we continue on, we should install Nodemon. This package will restart the server for us so that we don't have to keep doing that after every change.

Install Nodemon generally with this command (new terminal window):

```
sudo npm install -g nodemon --save-dev
```

1. Install Nodemon again in the folder you are using to build your application

   ```
   npm install nodemon --save-dev
   ```

   Note that Nodemon is a development dependency, it does not have to be installed in the final application, hence `--save-dev` will ensure that this does not happen.

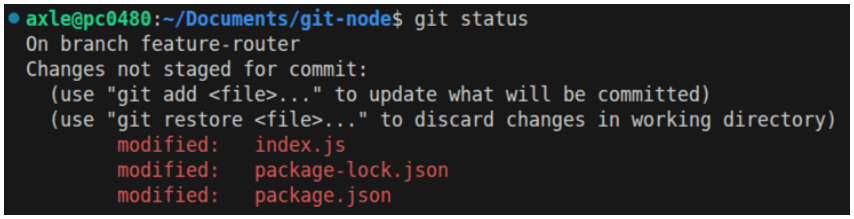
   With Nodemon installed, once you are in the directory just issue the command `nodemon` and the app will spin, you don't even have to point it to the file index.js.

2. However, it is customary to add a script that will solidify what Nodemon should do when we start the application. So open your package.json file and change the following lines:

   ```
   "main": "index.js",
   "scripts": {
     "start": "nodemon index.js"
   },
   "author": "",
   "license": "ISC",
   "dependencies": {
   ```

3. So now we can start the app using just `npm start` or `nodemon`**.** Also when we make changes in the future and save the file, the server will restart and accept our changes. Of course, if there is a syntax error in our code, it will report it as well.

4. If you do a git status now you should see several files colored red. Git has been tracking all the changes. In the next part we will *commit* them and add a message.



Part 05 – Merging the branches

After Part 04, start the application using the command `npm start`. Use Postman to post a new dummy user to the /employees endpoint. If you do not get an error, you are ready for the next step. It means that the changes we made by adding Router was successful. Based on this success it is now a good time to merge those changes back on to the main (master) branch.

1. While still on the *feature-router* branch, run the following command to make sure that Git knows which files were changed:

```
git add .
```

Note the last character is the period character, it must be used.

2. Let everyone know what changes you made:

```
git commit -m "Add Express Router, initial routes and a POST route"
```

3. (optional) To know which branch you are on, just type:

```
git branch
```

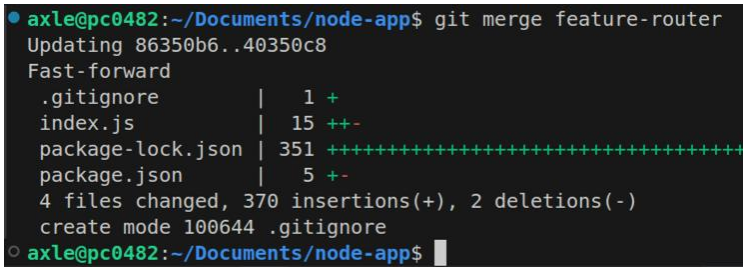
You will see a list of all your local branches. The branch you are currently on will have an asterisk (*) next to it and will usually be highlighted in green.

4. Once you are sure the branch is working as intended, it is time to merge it with the main branch. First switch to the main branch:

```
git switch master
```

5. Now the merge command

```
git merge feature-router
```



6. At this point you could verify which branch you are on, see #3. Once you are satisfied, just delete the feature-router branch:

```
git branch -d feature-router
```

After this you should now have just one branch, the *master*. You can also delete remotely.

Part 06 – Typical Git commands

1. If you are on a unix-based system like Ubuntu, the *ls* command will give you a list of all files in a directory. There is also the *history* command in Linux. To see a history of all the Git commands, from the POV of the OS, you could do this:

```
history | grep git
```

This command search's the terminal's history to return every command with the word "git".

2. Back to Git and your project's history. The following command doesn't show what you typed; it shows what you committed to the code base:

```
git log
```

If the history is very long and you want to get out, just type the letter **q**. For browsing, use arrow keys and for paging use spacebar. You could also use commands like `git log -n 3` which will show the three most recent commits. *If you are stuck and cannot get out of a long git list, type "q" for quit*.

3. (optional) If you accidentally delete a branch or "lose" a commit, `git log` won't show it, but `git reflog` will:

```
git reflog
```

If you wanted to pipe the output of any of these commands, you could try commands like this:

```
git log --pretty=format:"%ci %s" |  tee gitlog.log
```

That command will output only the dates and commit messages to the file called gitlog.log.

4. If you accidentally made a mistake in one of the files, say I added these ellipses and did not mean to do that:

```
app.listen(port, function(){
      console.log("Listening… " + port);
});
```

5. Just run the following command

```
git restore index.js
```

The ellipses will just disappear.

Here are a few tools we did not practice but will come in handy as your project grows:

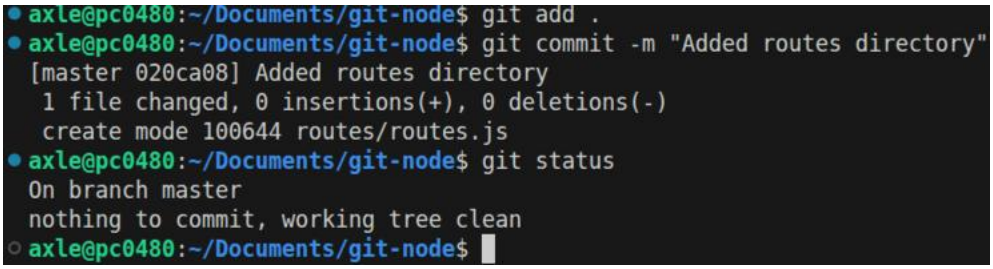
git log --graph To see where your branches diverged and merged.
git restore "I hate these changes I just typed, make them go away."
git reset --soft "I committed too early and need to add one more thing."
git stash "I'm not ready to commit, but I need to switch branches NOW."

Part 07 – Advanced Git Commands

In this part we will practice *rebasing* and *squashing*. Depending on how many times you commit, you may end up using the same phrases in your message. You may see several "fix," "oops," or "typo". We will practice squashing these. This command will combine multiple small, messy commits into one clean, professional feature commit before you "submit" it.

1. In VS Code and in the current directory, create a new folder called routes and inside of that directory, create a new .js file called routes.js. Now run the following Git commands in the terminal:

```
git add .
git commit -m "Added routes directory"
git status
```



2. The first line of the routes.js file, will be a variable pointing to a function, we have to do this in order for other files in our application to know that the routes file exists.

```
module.exports = function(){};
```

3. Next we will CUT the two functions from our index.js file and paste into this routes file:

```
module.exports = function() {
  router.get('/', (request, response)=> response.send('hello from skillsoft'));
  router.post('/addemployee', (request, response) => {
    let empName = request.body.empName;
    let empPass = request.body.empPass;
    response.end(`POST success, you sent ${empName} and ${empPass}, thanks!`);
  });
};
```

Place the `get()` and `post()` functions inside of the curly braces of the exported function.

4. However this file does not have access to `router`, but we can pass `router` to this file when we call the exported function from index...coming up:

```
module.exports = function(router){
  router.get('/', (request, response)=> response.send('hello from
skillsoft'));
  router.post('/addemployee', (request, response) => {
    let empName = request.body.empName;
    let empPass = request.body.empPass;
    response.end(`POST success, you sent ${empName} and ${empPass},
thanks!`);
  });
};
```

5. Switching to Git, run the same commands as in #1 but with a slightly different message:

```
git add --all
git commit -m “Completed routes file”
git status
```

6. Back in the server file (index), we have to let it know where to find routes.js, so create a variable and point it to the new routes.js file inside of the routes directory.

```
const express = require('express');
const routes = require('./routes/routes');
const router = express.Router();
routes(router);
const port = 8000;
```

Remember we had cut the two route functions, so this file should be very short.
Test the Node application to make sure it works, `npm start` will work.

7. Switching to Git, run the same commands as in #4 but with a slightly different message:

```
git add --all
git commit -m "Completed index file to work with routes"
git status
```

```
axle@pc0480:~/Documents/git-node$ git add .
axle@pc0480:~/Documents/git-node$ git commit -m "Completed index file to work with routes"
[master f386d90] Completed index file to work with routes
 1 file changed, 2 insertions(+)
axle@pc0480:~/Documents/git-node$ git status
On branch master
nothing to commit, working tree clean
```

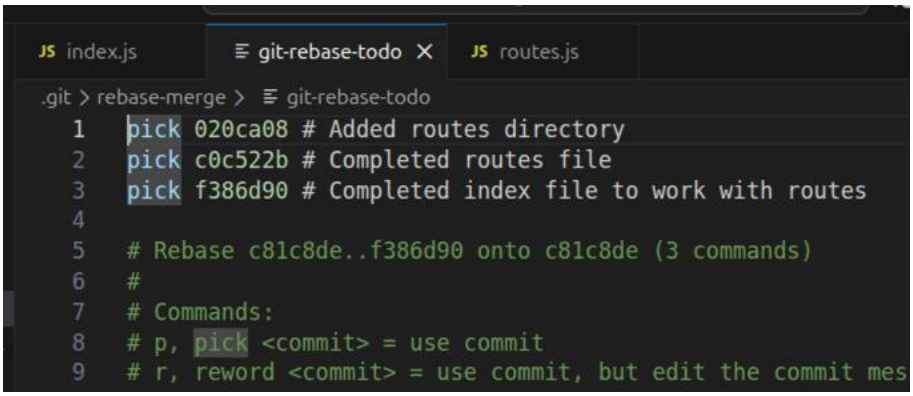
8. Before we run the rebase command, lets take a look at the last four commits, run this command:
   `git log --oneline -n 4`
   The – -oneline command is a condenser

```
git-node:$ git log --oneline -n 4
ea739d6 (HEAD -> master) Completed index file to work with routes
536d3b6 Completed routes.js file
aa8b0f7 Added a routes directory
35a187d Add Express Router, initial routes and a POST route
git-node:$
```

9. Now its time to *rebase*, so run this command in the terminal window while still in VS Code. This command will clean up your messages. Rebase allows the developer to edit any of the commit messages:

```
git rebase -i HEAD~3
```

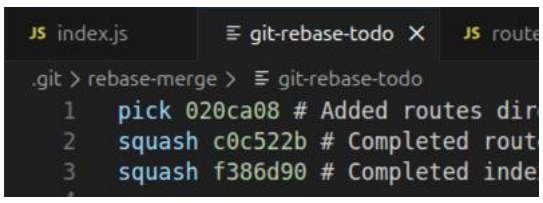
   A new editor tab in VS Code will open up. The `-i` means opens the "Interactive" menu.



Note, this is VS Code. Remember we had configured Git to use VS Code as our editor.

In Git, the tilde (~) is a relative pointer that tells Git to move backward through previous commits. Specifically, `HEAD~3` means "start at the current commit (HEAD) and go back 3 steps into the past." If you aren't sure how many commits to go back, run `git log --oneline` first. Count how many commits you want to change, and use that number after the tilde.

10. Now at the very top the first three lines start with “pick”. Leave that **first** line as is but change the second and third to just “`squash`” or just “s”:



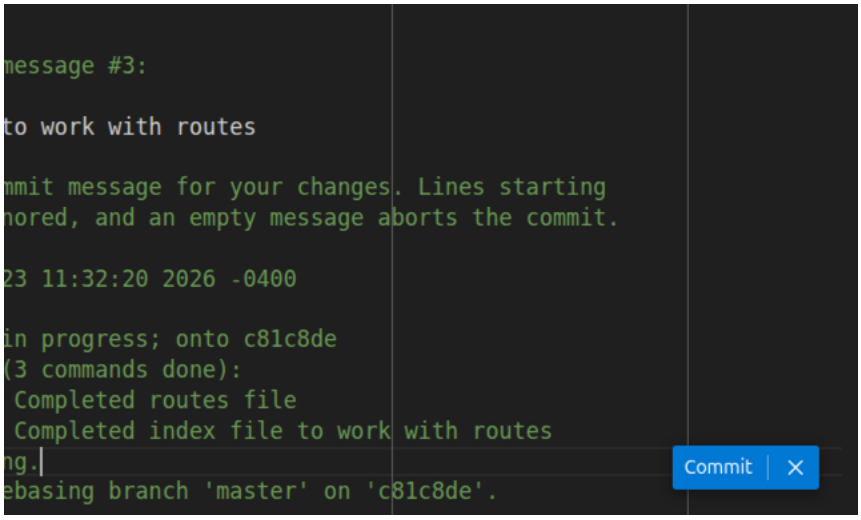
Now just close the new editor tab and you will see a *Commit* blue button some where on that editor window.

```
git-node:$ git log --oneline -n 4
7d2d4ea (HEAD -> master) Added a routes directory
35a187d Add Express Router, initial routes and a POST route
c7daa7f Added .gitignore file
bfb4144 First commit of new Node-based app
```

Compare this image to the one in #9, the two lines with “squash” are gone.

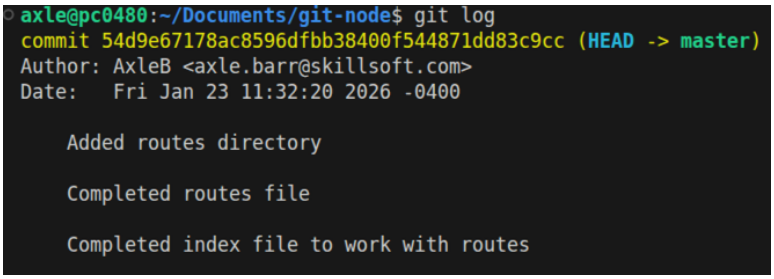
11. Press the *Commit* button to save the changes.

    One more useful Git command is **git show**. Use it any time to display a history of all activity in the directory.



You could always pipe the git show output to a file like this:
`git show > show.log`

12. To test that this squash worked, run a `git log`:



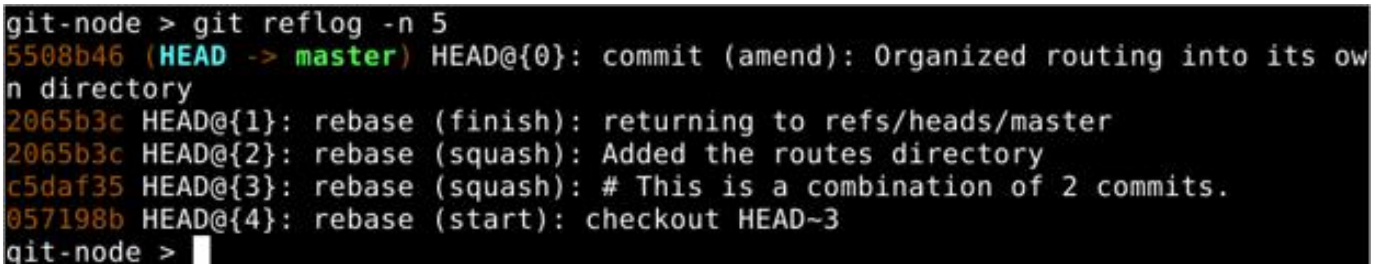
    Basically these three commits were rolled up into one commit.

13. It is possible at this point to want to change (edit) the last message. For that just run this command:

```
git commit --amend -m "Moved routing to its own directory"
```

    You can verify with `git log` as you did in #9. Also, `git show -1` will display the last commit.

14. The last command ran and the last commit was changed:



15. If you want you can compare this output to what we had before:

```
git-node > git reflog -n 5
2065b3c (HEAD -> master) HEAD@{0}: rebase (finish): returning to refs/heads/mast
er
2065b3c (HEAD -> master) HEAD@{1}: rebase (squash): Added the routes directory
c5daf35 HEAD@{2}: rebase (squash): # This is a combination of 2 commits.
057198b HEAD@{3}: rebase (start): checkout HEAD~3
3301e37 HEAD@{4}: commit: Completed index file to work with routes
git-node >
```

15. At this point if you did a plain `git log`, this is what you will see:

```
commit 5508b46e1111a362fdbf27ba659b53ad8c7e2908 (HEAD -> master)
Author: AxleB <axle.barr@skillsoft.com>
Date:   Fri Oct 23 11:09:41 2026 +0000

    Organized routing into its own directory

commit 08d7ac6306c6287e574edb2680cbcd4007a21be0
Author: AxleB <axle.barr@skillsoft.com>
Date:   Fri Oct 23 11:02:18 2026 +0000

    Added Express Router, initial route adn POST route

commit b59a214d43860547b6f1a222049e0470958d8891
Author: AxleB <axle.barr@skillsoft.com>
Date:   Fri Oct 23 10:51:18 2026 +0000

    Added .gitignore file

commit 4ab4a17dee3f43857ea30170b9530a758571c066
Author: AxleB <axle.barr@skillsoft.com>
Date:   Fri Oct 23 10:38:24 2026 +0000
```

So with a `git log` you don't even see the squashed lines or commits.

Part 08 – Git Tagging

In simple terms, Git Tagging is like creating a "save point", a snapshot in time. They are static points in your repository's history that you think are important. Usually, tags are used for software releases.

If a commit is a regular "Save" then a tag is like a "Milestone", a permanent label you attach onto one of those saves. Commits happen frequently but tags are rare, reserved for that special occasion.

Commits have a unique ID, and hard to remember (e.g. a1b2c3d). A *tag* gives that specific point in time a more readable name, like **v1.0** or release-candidate. Try to follow the MAJOR.MINOR.PATCH format:

Major: Breaking changes (e.g., 2.0.0).
Minor: New features, but backwards compatible (e.g., 1.1.0).
Patch: Backward-compatible bug fixes (e.g., 1.1.1)

1. Lets tag the current code as version 1.0.0

   ```
   git tag -a v1.0.0 -m "Release version 1.0 stable"
   ```

   This is called an annotated tag. To create a lightweight tag, without the message, it will be: `git tag v1.0.0-lw`

2. For a list of tags on the current project, just type:

   ```
   git tag
   ```

Once you learn about the GitHub part, you can create a workflow that includes *tagging*, *pushing* and *pulling* from the remote server

3. Once you see a list of tags you can examine just one by:

   ```
   git show v1.0.0
   ```



The following commands will not work until we create a repository on GitHub.com
Research these: `git rev-parse v1.0.0`, `git show-ref –tags` and `git log --oneline --graph --all`

Do not try to perform these as yet, they are meant for when we connect to a remote repository.

4. You can push to the remote repository, any tag

```
git push origin v1.0.0
```

   To push all tags it is `git push origin --tags`

5. You can checkout any tag into a detached HEAD state:

```
git checkout v1.0.0
```

This does NOT create a branch and any changes in this state may be lost. If you want to create changes to a checked-out tag, it is best to create a new branch. This can be done with one command, see below:

6. If you need to make changes or fix a bug starting from a specific tag, you should immediately create a new branch from that tag:

```
git checkout -b new-feature v1.0.0
```

   Here, *new-feature* is the name of the branch

7. Here is how you can delete a tag locally:

```
git tag -d v1.0.0
```

   For a remote delete, it will be: `git push origin –delete v1.0.0`

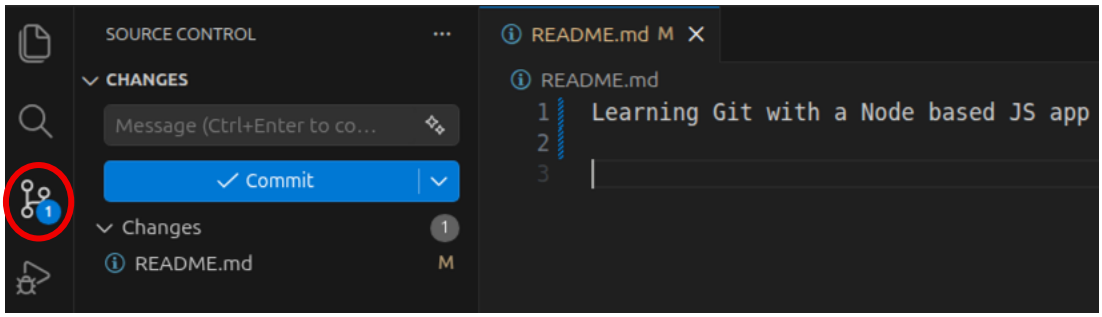
Part 09 – VS Code Source Control

Visual Studio Code has integrated source control management (SCM) that lets you work with Git and other version control systems directly in your editor.

The integrated source control interface provides access to Git functionality through a graphical interface instead of terminal commands.

Changes you make in the VS Code interface are synced with your command-line Git operations, so you can use both the UI and terminal as needed. The source control interface works alongside the command line rather than replacing it.

1. Make a change in the README.md file and immediately you will notice an icon change on the left side of your IDE. This means that SCM has detected a change:



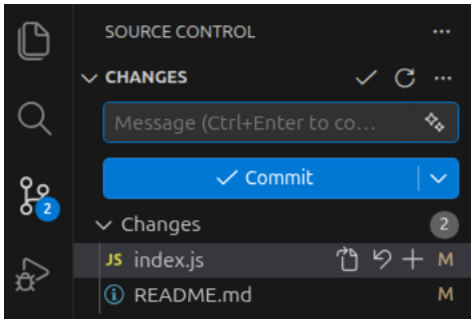
   If you click on the circled part, the Source Control panel will open instead of the file system Note, if you do not see an immediate reaction in the icon, click the icon then next to CHANGES there is a refresh icon, click it to pick up the changes.

2. Click on the File icon to open the explorer window and click on the index.js file to open that file in the editor window. Make a small change:

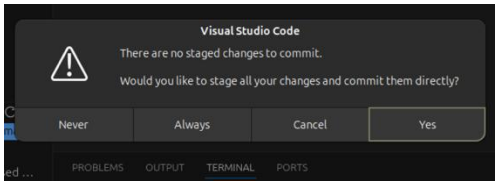
```
app.use(express.urlencoded({extended:false}));
  app.use('/', router);
  app.listen(port, function(){
  console.log("Listening on..." + port);
});
```

   Now you should see a 2 instead of a 1. We have now made two changes.

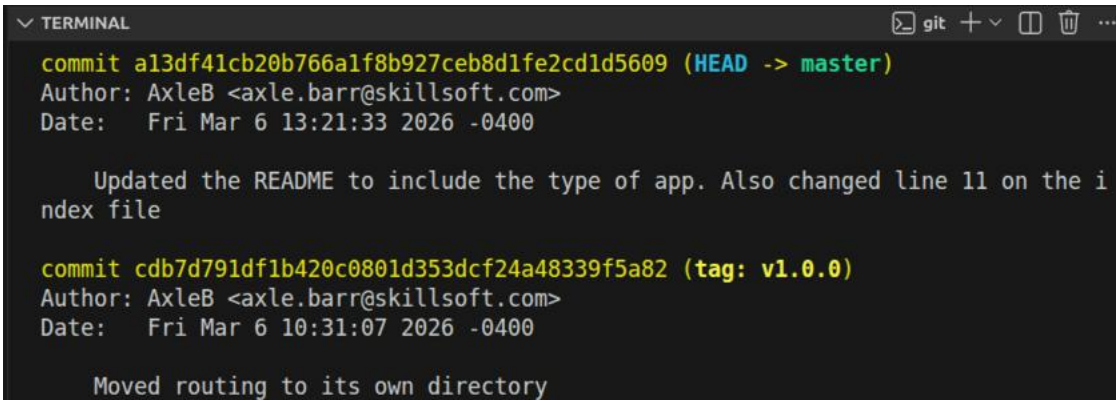
3. If you now click on the SCM in VS Code it will show the two files that were affected:



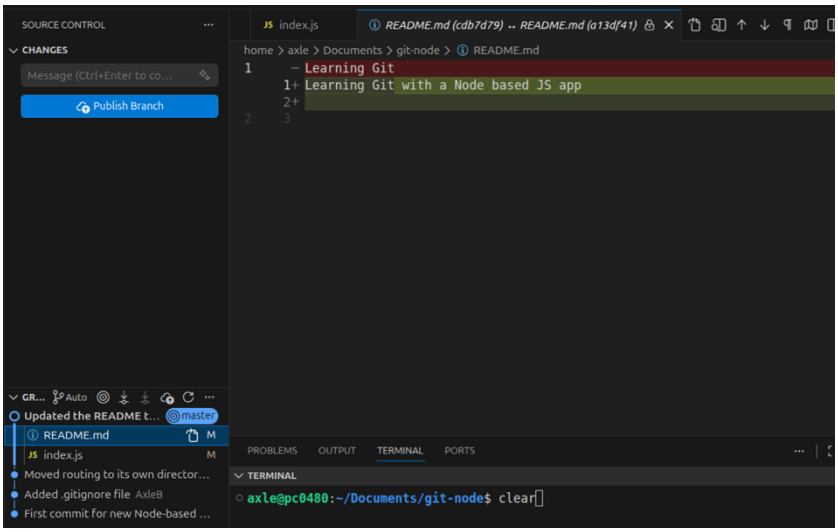
4. Notice the blue *Commit* button. This will of course commit the changes. Just above the Commit button there is a box to add your message. Add a message, for example "Updated the README to include the type of app. Also changed line 11 on the index file". Once you click Commit, VS Code will then ask for your staging instructions. Just hit "Always" here:



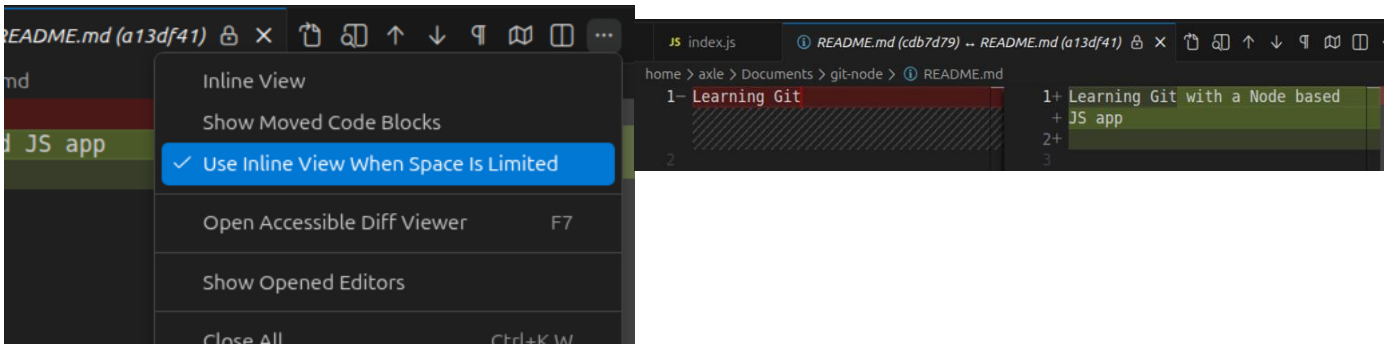
5. If you now do a **git log**, you will see the last commit with the message from #4:



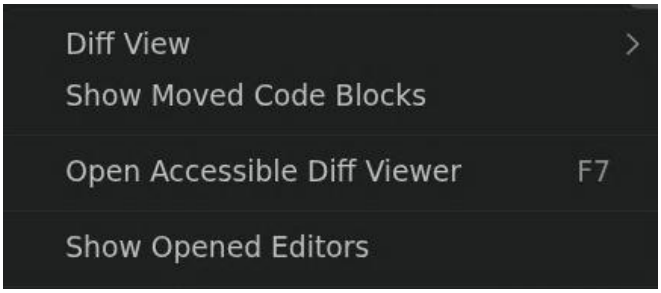
6. If you ever wanted to see the changes you made to a file, just click on that file from the version control panel. For example, if we click on the README.md file, the changes will come up in the editor window:



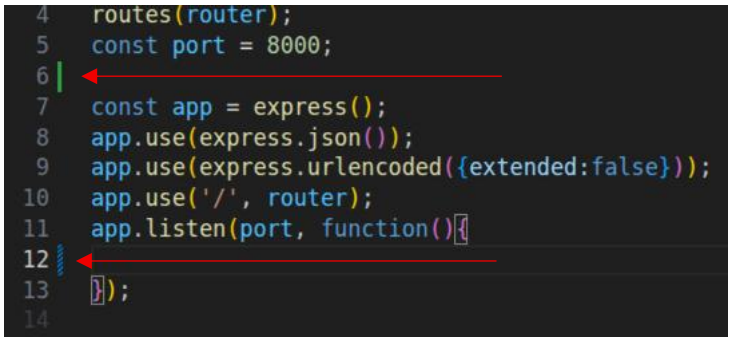
7. You can get a diff view if you click on the ellipses on the top right of VS Code and toggle the "Use inline View When Space Is Limited" option



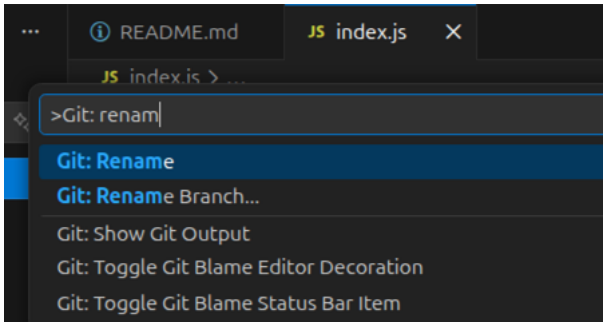
UPDATE: this has now changed in the latest release of VS Code. Use the Diff View now:



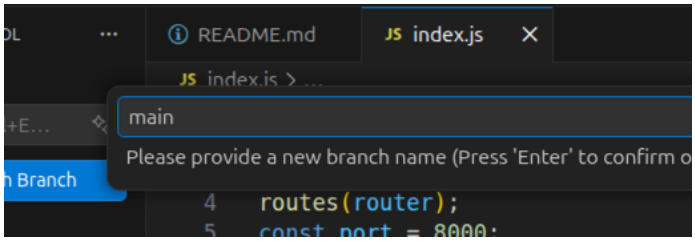
8. There are also gutter indications. In the index file, I added a blank line at position #6. Notice the green bar next to the #6. Also I remove the console.log() line. Again notice the blue hazed line.



9. The final task in this section, is to rename the branch we are on, using VS Code UI. We can do this via the Command Palette. In Linux hit Shift + Ctrl + p. Once the window opens, start typing *Git: renam* and from there you can choose your action. So in this case I will chose *Rename Branch*:



10. I then typed in “main” and hit the enter key:



Verify at the bottom left of the VS Code interface that you are now on the *main* branch.

Part 10 – Remote Repository

This section assumes that you have or will get a GitHub.com account. It is completely free to sign up and get a basic account on the site. From here on, *local* refers to your own computer/VM/sandbox and *remote* refers to your personal location on GitHub.com.

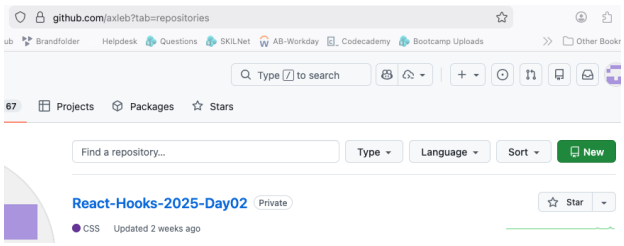
To get a local repository up to a platform like GitHub.com, we use a process called *pushing*. Just like copy/paste, this creates a perfect copy of the project, including its entire version history, on the remote location.

Before we can *push* (save) our git-node project, we should create a place for it on the remote site first. This is as simple as clicking on the green New button and following the instructions, on the site itself. For projects, you can start from the GitHub end and then do a *pull* instead of a *push*.

Note, there are several ways of interacting wit GitHub.com or any remote repository, this document talks about one approach.

1. We can now make the connection between the Git on our local machine to GitHub. Start from the GitHub end, log in and begin by creating an empty repository:

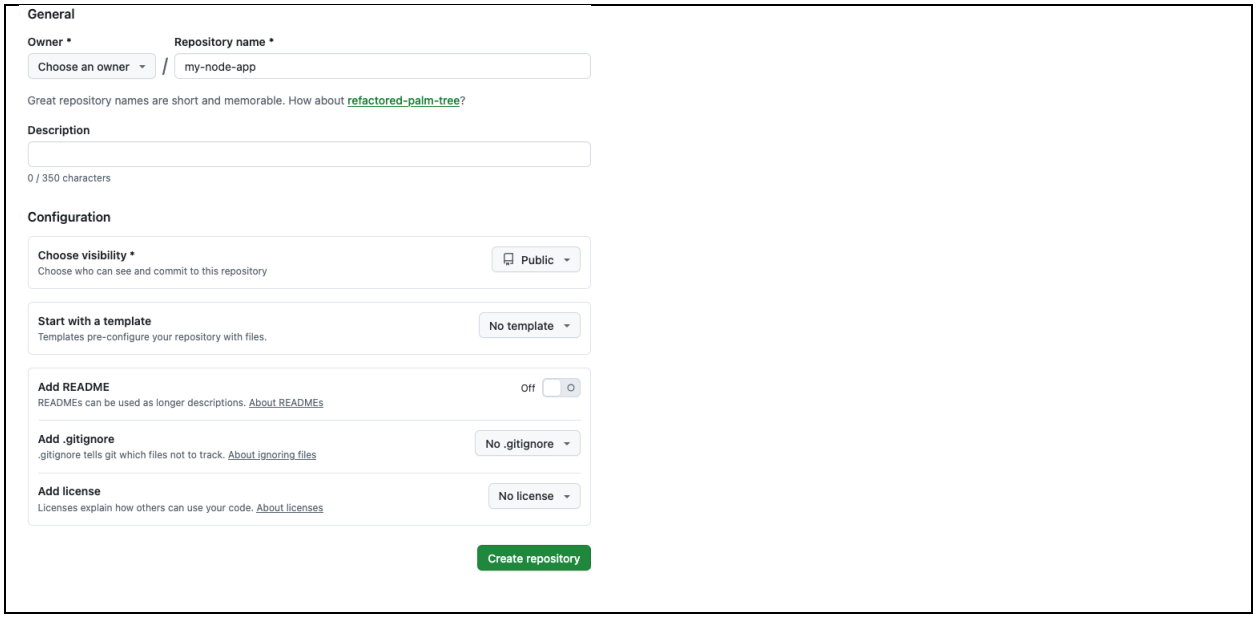
   Once you are on github.com, go to the *Repositories* tab and on the top right there should be a button labeled as **New**, click it to begin the new repository process.

   Repository name: Give it a name (e.g., **git-node**).
   Optionally you can give it a description.

   Settings: Keep it Public and do NOT initialize with a README, license, or .gitignore (since you already have these locally). If you have access to multiple repositories, you will need to choose the appropriate one via the “Owner” selection box.

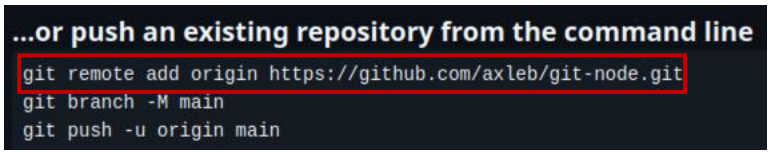
   Click the green *Create repository* button



After this you will get a long page with various instructional material. Do not make any changes here, the `push` command will populate the empty remote directory for you.

Note: for Node based projects, your entire project is not stored remotely. Only a skeleton is stored. That skeleton has a package.json file in it. This file tells the project to download all the supporting libraries, files, modules and other packages in order to restore the skeleton to an actual program that you can execute. For Node applications the command to get this started is `npm install`. Once you download a Node based project folder, just navigate to that folder and execute the `npm install` command against that folder. The app will rebuild itself.

2. We are ready to copy our project to GitHub.com. If you are on the page right after creating the new repository, look for the instruction involving the `push` command, it should be the last section of that page. Copy the URL that will be used to transfer your project, it should look something like this:



You only need to copy the first line, the one in the red box.

This line will establish a relationship between your directory and the remote GitHub.com repository you just created.

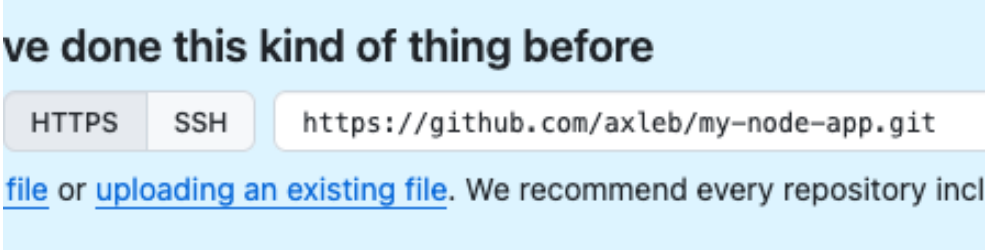
Remember also that I changed my local Git system to use **main** and not **master** as the default repository name. That change will determine whether you get an error or not.

3. Back to your local computer (or VM/Sandbox) add the text in green below and then paste the command you just copied from github.com. This is what it looks like on my machine. In your case it won't be *axleb*, it would be **your** GitHub name:

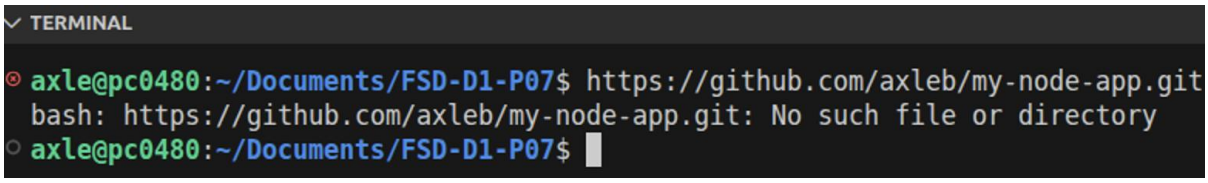
```
git remote add origin https:github.com/axleb/my-node-app.git
```

This is the username you chose when you created an account on GitHub.com.

It is also available on GitHub.com, once the second HTML interface page opens:



4. This is the response in the VS Code terminal window, after running the command in #3



5. (optional) Check the name of your branch, if it is `master` lets change it to `main` for modern standards:

```
git branch -m master main
```

Remember the command `git status` will tell you the current branch. Verify with `git branch`. If you completed Part09, above, this would have already been done.

6. (Optional) If you get an error about the hostname and access rights, lets make sure that git knows which protocol it is using. Type the following command:

```
git remote -v
```

If you see an image like this below, all is well:



7. If you are still getting errors after you run the command in #3, you can try the following commands. Let's reset, so run the following commands in sequence:

```
git remote remove origin
git remote add origin https://github.com/axleb/my-node-app.git
```

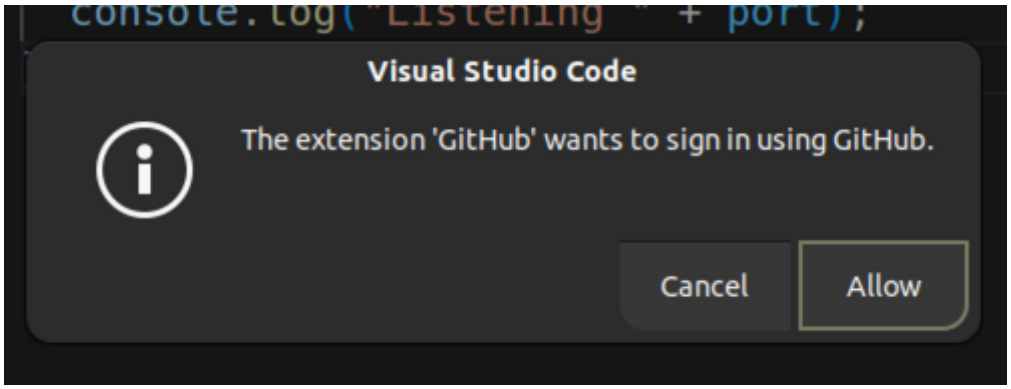
Once you are error free, you can continue with #8 below.

8. Now we can do the push, the command is:

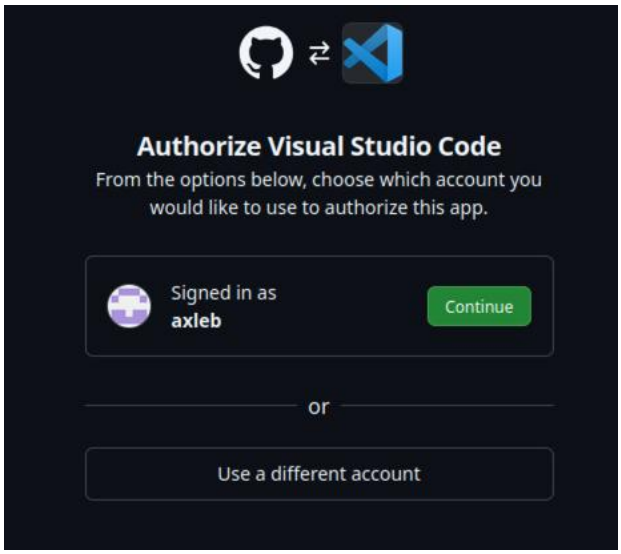
```
git push -u origin main
```

Alternatively you can try to use VS Code and click the *Publish Branch* in the **Source Control** panel. The -u is like a memory bank, it allows tracking of the local branch.

9. You may see a message pop up like this:

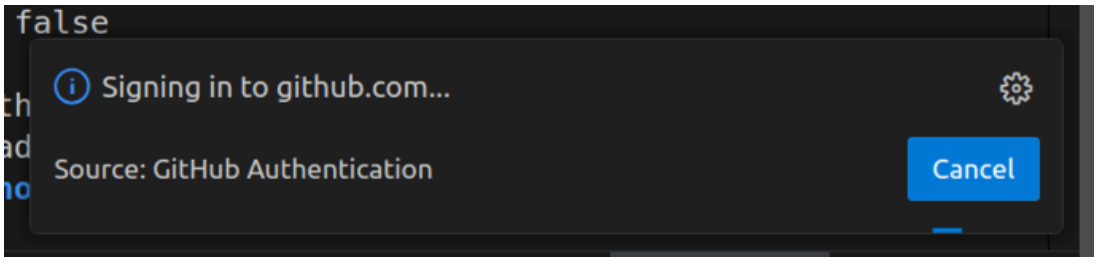


Click `Allow`

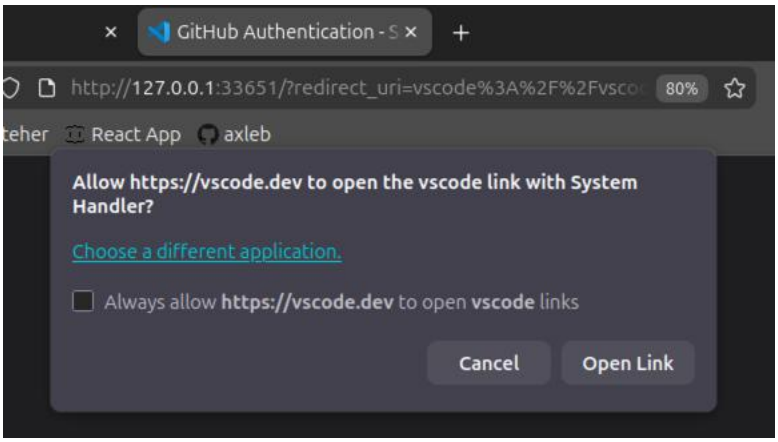


Click `Continue` here.

Then you will see the signing process begin:



Click on `Open Link` when this modal dialog shows up:

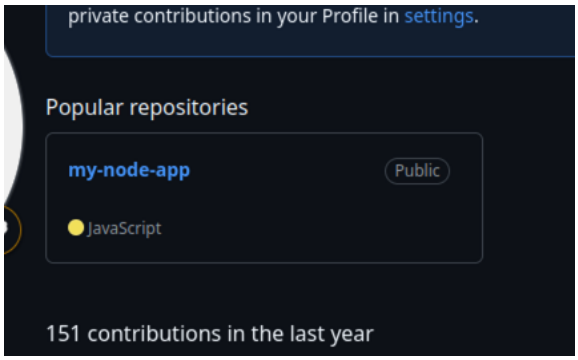


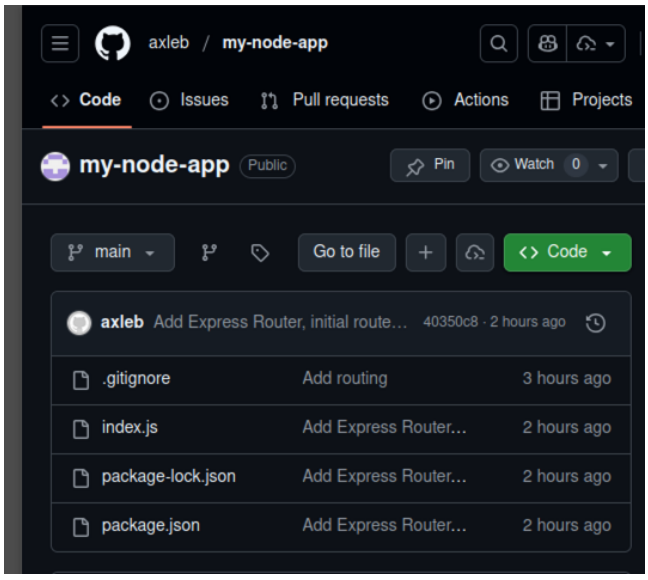
10. If you got to this point, you should see something like this in the terminal window:

```
axle@pc0482:~/Documents/node-app$ git push -u origin main
Counting objects: 100% (17/17), done.
Delta compression using up to 2 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (17/17), 12.06 KiB | 6.03 MiB/s, done.
Total 17 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/axleb/my-node-app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Refer to the appendix for possible errors when running this command:

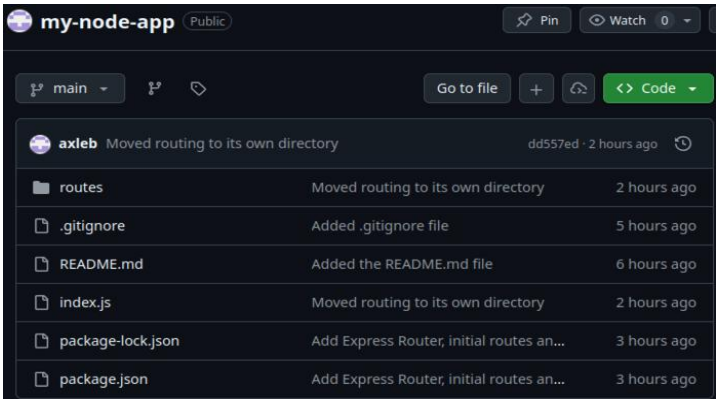
11. Now if you go to your GitHub account, the files for this NodeJS project should be there:





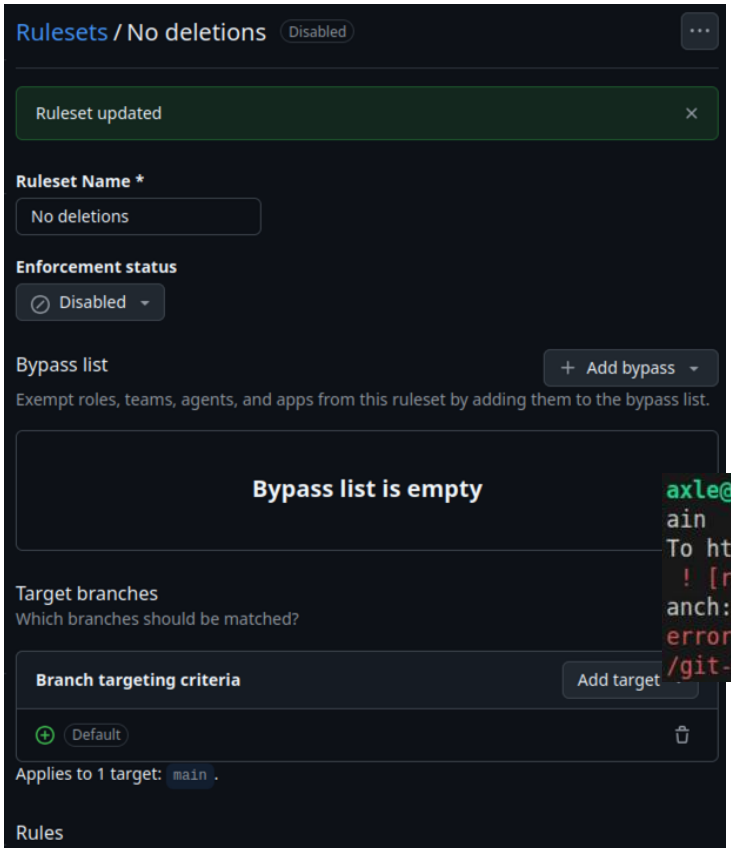
You may have to exit the Create page and go back in to your new repository to see the files.

12. Click on the repository and it will take you into the area where the files are kept:



Notice that the node_modules directory is not in the remote location.

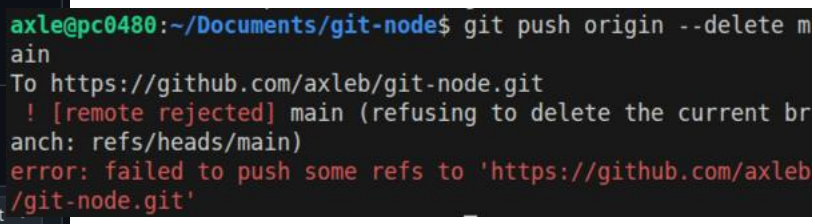
13. (Optional) If you want to protect the main branch follow the instructions below.
    If you wanted to protect your main branch, go to *Settings* then *Branches*. You can choose from classic to fine tuning a rule. In this case I chose *New branch ruleset*. Give it a name like “No deletions”. Click on *Add target* and add the default branch. Finally click on the *Create* green button bottom left of that page.



Now if you want to delete the main branch on the remote from the local you can try this:

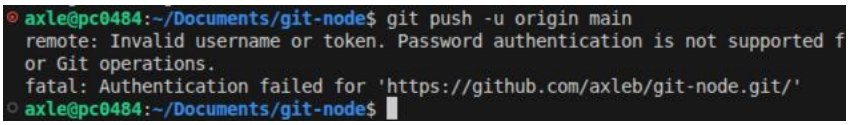
git push origin --delete main

You would get a message something like this:



Appendix A – PAT Error

If you got an error like this when you try to push, it probably means that GitHub is looking for a Personal Access Token (PAT).



Think of a PAT as a special password just for your terminal that doesn't trigger 2FA.

Go to your GitHub account on the web: Settings > Developer Settings > Personal access tokens > Tokens (classic).

Click Generate new token.

Give it a name (like "Bootcamp Demo") and check the box for repo (this gives it permission to push code).

Copy the token immediately! You won’t see it again.

Go back to your terminal and run the push command again:

```
git push -u origin main
```

When it asks for a Password, paste that token instead. (Note: The terminal won't show any characters while you paste, just hit Enter).

If you are having authentication problems you can unset the current credentials and try again:

Unset the current helper

```
git config --global --unset credential.helper
```

Tell Git to stop trying to be smart and just ask you

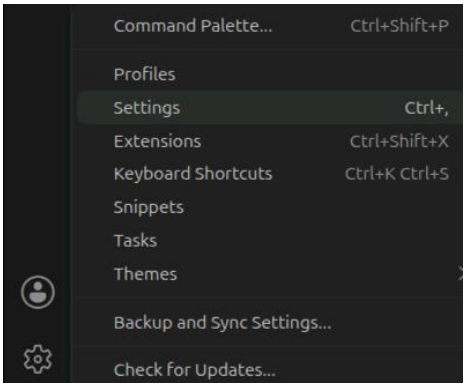
```
git config --global credential.helper ""
```

Try to push again

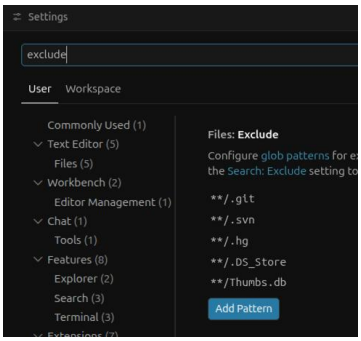
```
git push -u origin main
```

Appendix B – Showing the .git Folder in VS Code

The .git folder is hidden by default. To show it in VS Code File Explorer, go to the settings for VS Code. You can use CTRL+SHIFT+P or use the settings cogwheel icon bottom left of VS Code:



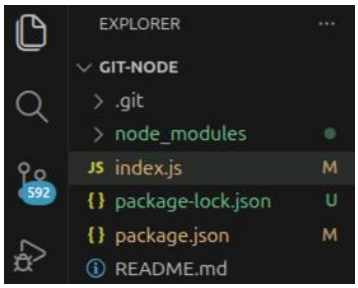
Once in settings, type into the search bar "exclude":



Now just use your mouse to hover over the line with **/.git and on the right side an "X" will show up. Click the X to remove this setting for .git files.



The .git folder will now show:

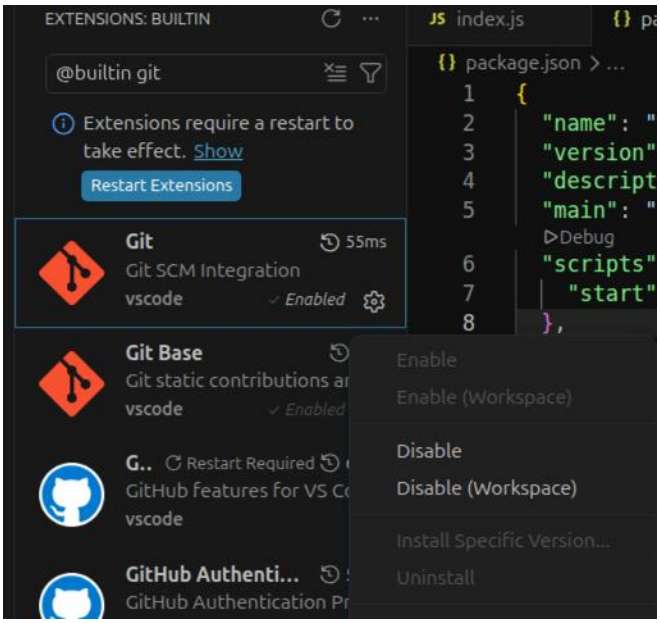


Appendix C – Fixing Source Control Icon Not Refreshing

If the icon is not updating when a change is made, sometimes disabling and then enabling the built in Git extension in VS Code may help.

Here are the steps I took:

1. Open the Extensions tab using:
   CTRL + SHIT + X
2. Search for @builtin git
3. Right on the Git Base extension. Click “Disable”, the tab will close, open it back up and click “Enable”



You may have to repeat these steps for the Git extension also, but usually Git Base will handle both extensions.

Appendix D – Fixing Source Control Icon Not Refreshing

The ten most common local Git actions include initializing repositories, staging and committing changes, managing branches, and iterating to historical events. These tasks help you managing a local Git repository. Consider them a core set of workflows for tracking, saving, and navigating your code.

1. Check Repository Status: git status
2. Stage Changes: git add <file> or git add .
3. Save a Snapshot (Commit): git commit -m "Your message"
4. Create and Switch Branches: git checkout -b <branch-name> or git switch -c <branch-
5. Switch Between Branches: git checkout <branch-name> or git switch <branch-name>
6. View Commit History: git log --oneline or git log --graph
7. Review Uncommitted Changes: git diff
8. Merge Branches: git merge <branch-name>
9. Undo Unstaged Changes: git restore <file>
10. Pause Work Temporarily: git stash

Appendix E – Shorten the Linux Prompt

To shorten the bash prompt to just a $ you can do this: **PS1='$ '**
Note: there is a space between the $ and the ending apostrophe

Another option is: **PS1='\u:\W\$ '**

Appendix F – Common Errors

1. If you see something like this:

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
git-node$ git commit -m "Added the README.md file"
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

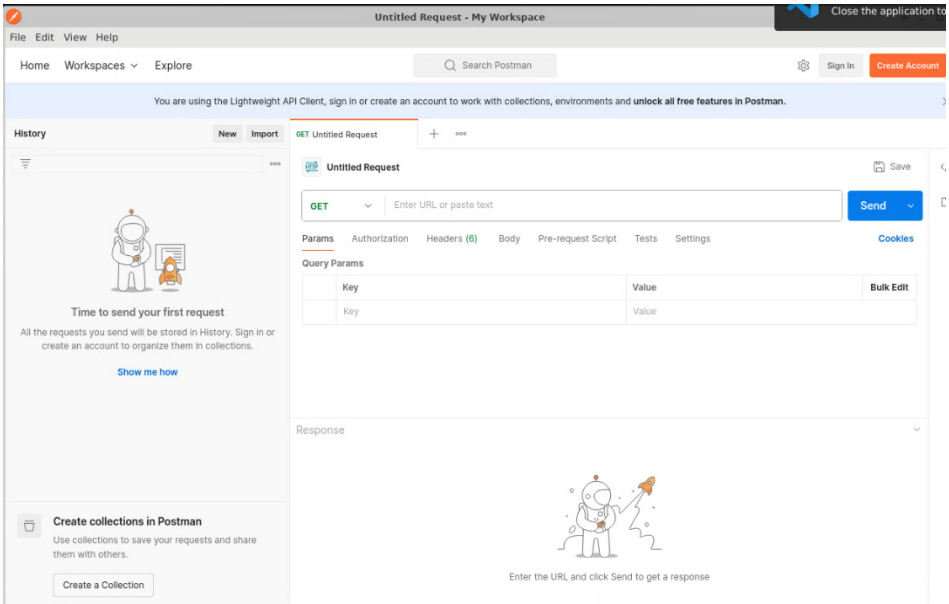
nothing added to commit but untracked files present (use "git add" to track)
git-node$
```

It means that you tried to commit the changes without adding the file first, so make sure you run this command before the commit:

`git add README.md`

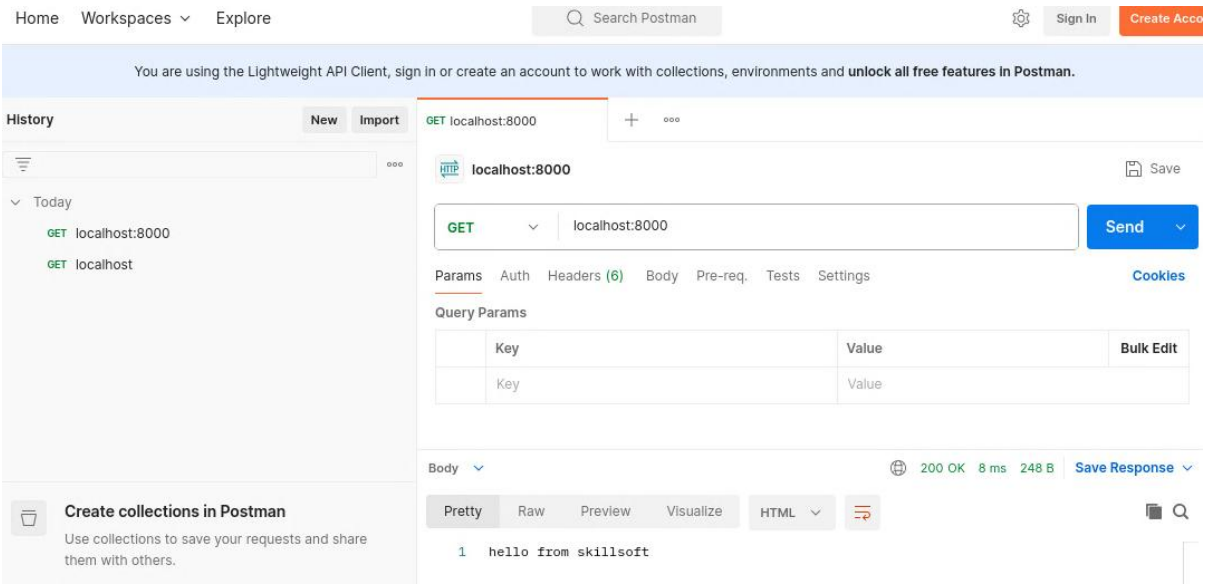
Appendix G – Postman Setup

1. If you see something like this, it means that Postman is ready to test your end points:

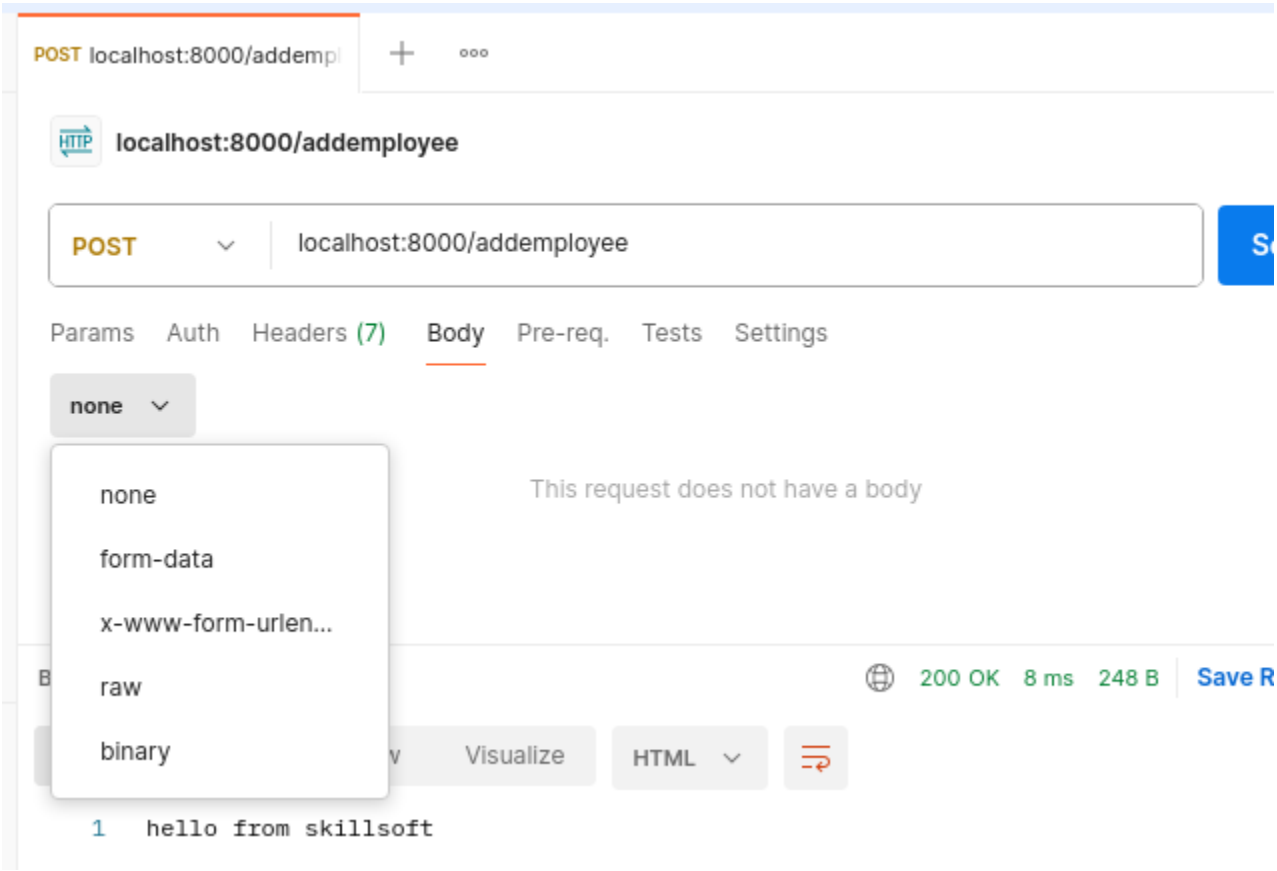


It means that you just opened the application and you can either create an account or just start using it.

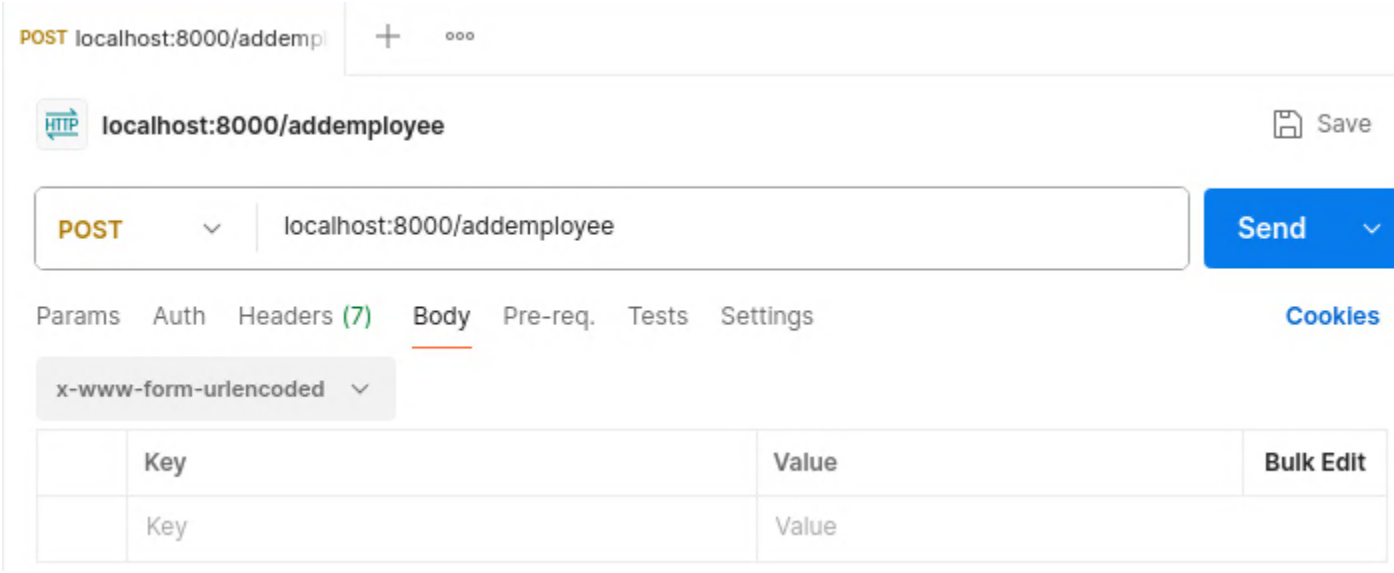
2. To start using Postman just do a simple test to the root endpoint and you should get the same response you got from your browser:



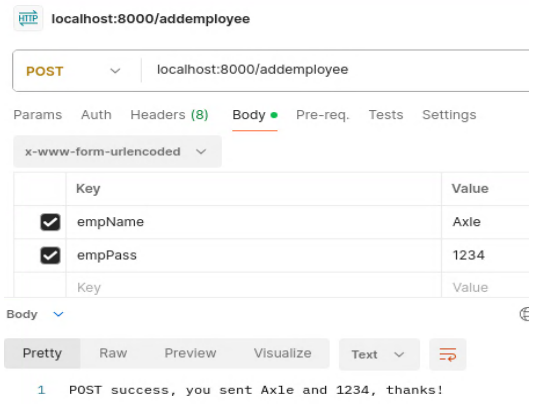
3. If you see “hello from Skillsoft” then try the POST request. First, change the GET request to a POST request. Click on the “Body” tab and then click the “none” drop-down arrow. Choose x-www-form-urlencoded:



4. If you followed the instructions in #3, this is what the screen should look like now:



5. We now need to add the key/value pairs that the code in index.js needs, so empname and emppass:



Now just click the “Send” button and the response section below should return what the code in index.js has.

Appendix H – git log VS git reflog

Feature	git log	git reflog
Tracks	Commits in the project history	Every movement of HEAD
Shared?	Yes, pushed to remote repositories	No, strictly local to your machine
Persists deleted work?	No (lost commits disappear)	Yes (recovers detached/deleted states)
Primary Use	Reviewing code evolution and authorship	Undoing mistakes and recovering "lost" work

In Git, **HEAD** is a pointer that represents your **current working position**. It is like a bookmark or cursor that tells Git which commit, branch or state it is looking at.

Usually HEAD points to the **name of the branch** you are currently on (e.g., master, main, or feature-login). When you make a new commit, that commit is added to the current branch, and HEAD moves forward along with it automatically.

In "Detached HEAD" state, HEAD can sometimes point directly to a **specific commit hash** instead of a branch. This happens if you check out an old commit directly (e.g., git checkout <commit-hash>). If you make changes and commit while in this state, they won't belong to any branch and can easily become lost if you switch away.